

MODULE 6 · SCHEDULING, RESOURCES & SCALING

Scheduling

How Kubernetes decides where a Pod runs

Every Pod needs a home — you can influence which one

The scheduler assigns every Pod to a node behind the scenes. Left alone, it just finds any node with room. Real clusters aren't that uniform — nodes differ in hardware, role, and failure domain — and sometimes you need a say in where a Pod lands.

Kubernetes gives you two different kinds of lever: rules that **attract** a Pod toward certain nodes, and rules that **repel** a Pod away from nodes unless it explicitly opts in.

BY THE END YOU CAN

- Explain the scheduler's filter-then-score model
- Pin a Pod with nodeSelector / node affinity
- Explain why a taint repels but a toleration only allows
- Read why a Pod is stuck Pending

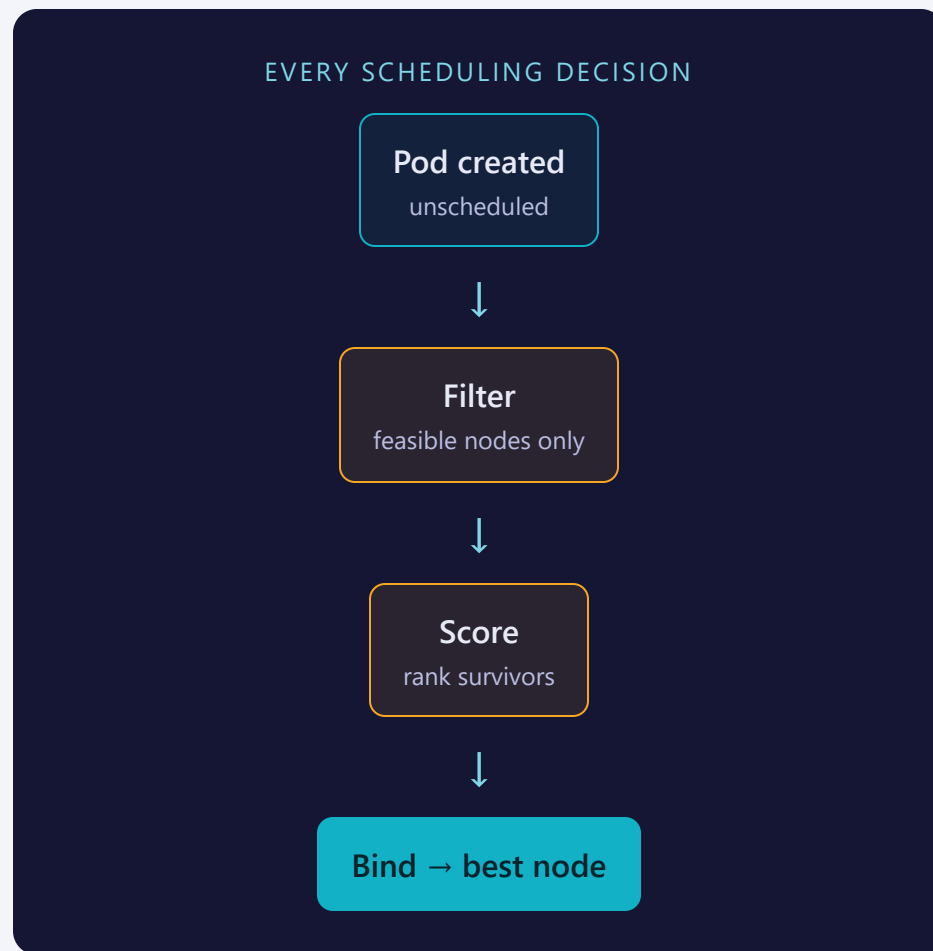
CONCEPT

Filter, then score — every time

For every unscheduled Pod, the scheduler runs the same two-phase decision:

- **Filtering** — drop every node that can't work: taints without a toleration, not enough CPU/memory, a nodeSelector or required affinity that doesn't match
- **Scoring** — rank what's left (preferred affinity, spreading, resource balance) and bind the Pod to the best one

Zero nodes survive filtering → there's nothing to score → the Pod stays Pending.



nodeName, nodeSelector, and affinity compared

MECHANISM	MATCH STRENGTH	NOTES
<code>nodeName</code>	Bypasses the scheduler	Hardcodes to one exact node — no rescheduling if it's down
<code>nodeSelector</code>	Required (hard)	Simple key=value match; no match → Pod stays <code>Pending</code>
<code>requiredDuringScheduling</code>	Required (hard)	Like <code>nodeSelector</code> , but with operators: <code>In</code> , <code>NotIn</code> , <code>Exists</code> ...
<code>preferredDuringScheduling</code>	Soft, weighted	Scheduler tries; a non-matching node is still acceptable

MENTAL MODEL

`nodeSelector` and `required` affinity **REQUIRE** a match to schedule at all. Keep that word in mind — taints and tolerations, next, work the opposite way.

Where placement rules live in the spec

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-node-affinity
  namespace: training
  labels:
    app: affinity-test
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 80
          preference:
            matchExpressions:
              - key: disktype
                operator: In
                values:
                  - ssd
                  - nvme
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/os
                operator: In
                values:
                  - linux
    containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

Both blocks live under `spec.affinity.nodeAffinity`: **required** (`kubernetes.io/os In [linux]`) must match to schedule at all; **preferred** (weight 80, `disktype In [ssd, nvme]`) only nudges the scorer — a plain `nodeSelector` with `disktype: ssd` would sit as a sibling of `affinity` and behave like the required block above.

Node changes are imperative, placement rules are declarative

IMPERATIVE — CHANGE THE NODE, NOW

```
kubectl label node node01 disktype=ssd
```

```
kubectl taint nodes node01 environment=production:NoSchedule
```

DECLARATIVE — CHANGE THE POD SPEC

```
nodeSelector:  
  disktype: ssd
```

```
tolerations: - key: environment ...
```

THE BRIDGE TRICK

Scaffold the Pod imperatively, then hand-add the placement fields before applying:

```
$ kubectl run web --image=nginx:1.25 \  
  --dry-run=client -o yaml > pod.yaml # then add nodeSelector / tolerations
```

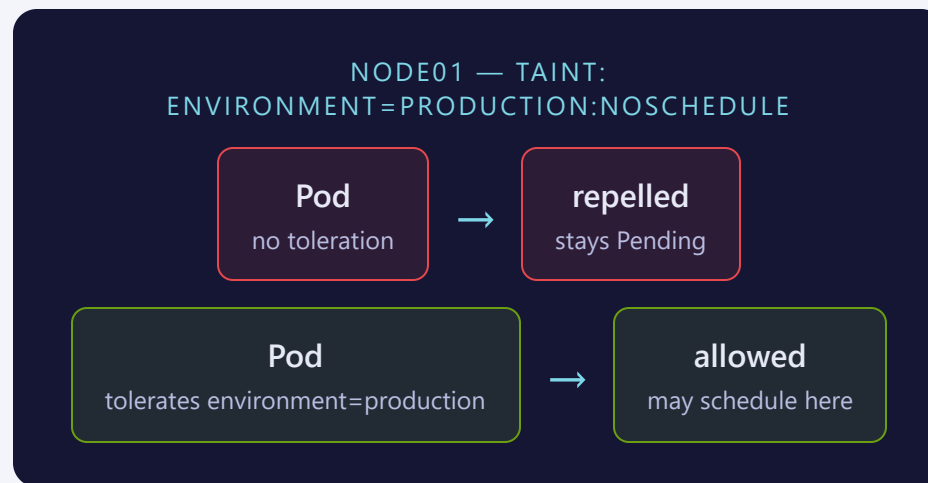
A taint repels; a toleration only allows

A **taint** lives on the node: "don't schedule here unless you say otherwise." A **toleration** lives on the Pod and removes that repulsion — it does not attract the Pod there.

- **Taint** key=value:Effect — set on the node
- **Toleration** — set on the Pod, must match key/value/effect
- NoSchedule — hard block for new Pods
- NoExecute — also evicts Pods already running there

ALLOW VS. REQUIRE

A tolerant Pod **may** land on the tainted node — it isn't guaranteed to. That's the opposite of nodeSelector, which requires the match to schedule at all.



A tolerant Pod, in full

```
apiVersion: v1
kind: Pod
metadata:
  name: taint-tolerant
  namespace: training
spec:
  tolerations:
    - key: environment
      operator: Equal
      value: production
      effect: NoSchedule
  containers:
    - name: nginx
      image: nginx:1.25
```

The toleration's key/value/effect must match the node's taint exactly — this Pod may now schedule onto a node tainted `environment=production:NoSchedule`.

SEE IT

A Pending Pod tells you why, if you ask



```
$ kubectl taint nodes controlplane environment=production:NoSchedule
$ kubectl taint nodes node01 environment=production:NoSchedule
$ kubectl describe pod taint-test -n training
Events:
  Warning   FailedScheduling    0/2 nodes are available: 2 node(s) had intolerated taint {environment: production}.
```

describe lists exactly which nodes were filtered and why — intolerated taint, unmatched affinity, insufficient resources. get pod only shows the phase.

Where people trip

- **Typo in nodeSelector.** No error, no warning — the Pod just sits Pending forever.
- **Toleration $\neq$ attraction.** It only removes a repulsion; pair it with affinity if you need that node specifically.
- **NoExecute evicts running Pods,** not just new ones — unless the toleration sets tolerationSeconds.
- **nodeName bypasses the scheduler entirely.** Node goes down, Pod down — no rescheduling.

Commands to keep at hand

```
$ kubectl get nodes --show-labels           # see node labels
$ kubectl label node NODE key=value         # attract with nodeSelector
$ kubectl taint nodes NODE key=value:Effect # repel with a taint
$ kubectl taint nodes NODE key=value:Effect- # remove taint (trailing -)
$ kubectl describe node NODE | grep -i taint # check taints
$ kubectl describe pod NAME                 # see FailedScheduling events
```

Tip: describe the Pod, not just get — Pending hides its reason.

RECAP

Scheduling in one breath

- Scheduler = filter feasible nodes, then score and bind to the best
- nodeSelector / required affinity **REQUIRE** a match — no match, no schedule
- Preferred affinity is a soft nudge, never a guarantee
- Taints **REPEL**; tolerations only **ALLOW** — they don't attract

HANDS-ON

Now run Lab 6.1
`labs/6.1/`

Next: [6.2 — Requests & Limits](#)